Clase 21 · Cifrado AES con OpenSSL entre cliente y servidor

Arquitectura Cliente-Servidor · Mtro. Carlos Alberto Román Zamitiz · jueves 5 nov 2026 · Tema 5 · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

Cifrado simétrico con la API EVP de OpenSSL (AES-256 en modo CBC con relleno PKCS#7) aplicado a un cliente y un servidor en C: llave en archivo, IV aleatorio, protocolo de tres send y tres recv. Las Clases 19 y 20 (29 oct y 3 nov) no se asistieron y su página está pendiente.

> **Fuente**
> NotebookLM (audio ACS-Clase21.m4a). Lo que no se pudo contrastar con el audio se marca como dudoso.

1. Relleno PKCS#7 en bloques de 16 bytes

- AES cifra por bloques de **16 bytes**; el relleno (*padding*) completa el mensaje al siguiente múltiplo de 16.
- Si faltan **k** bytes, se agregan **k bytes con valor k**. Ejemplos de la clase: «Hola Mundo» (10 bytes) recibe 6 bytes de valor 6; un texto de 12 recibe 4 de valor 4; uno de 11 recibe 5 de valor 5.

> **Examen**
> Si el mensaje mide **exactamente 16 bytes**, se agrega un **bloque completo de 16 bytes con valor 16 (0x10)** y el total pasa a 32.
>
> - Un bloque de 16 bytes impreso en hexadecimal ocupa **32 caracteres** (2 por byte).
>
> Error común: creer que el relleno son ceros, o que 32 caracteres hexadecimales son 32 bytes.

2. La API EVP de OpenSSL

- Cabeceras: `openssl/evp.h` y `openssl/rand.h`.
- **Cifrar:** `EVP_CIPHER_CTX_new()` → `EVP_EncryptInit_ex(ctx, EVP_aes_256_cbc(), NULL, key, iv)` → `EVP_EncryptUpdate()` → `EVP_EncryptFinal_ex()`, con aritmética de punteros para concatenar el bloque final.
- **Descifrar:** `EVP_DecryptInit_ex()` → `EVP_DecryptUpdate()` → `EVP_DecryptFinal_ex()`, con la **misma llave y el mismo IV**. Al terminar se libera el contexto.
- Llave de **32 bytes (256 bits)**; IV de **16 bytes**.

> **Examen**
>
> El profesor mostró los dos códigos lado a lado: «Comparen los dos textos… encrypt init, decrypt init; encrypt update, decrypt update; encrypt final, decrypt final.»
>
> Error común: descifrar con un IV distinto al que se usó para cifrar; el descifrado falla.

3. Llave e IV

- La llave simétrica va en un archivo, `llave.key`; se abre con `fopen("llave.key", "rb")` y se lee con `fread` a un arreglo de 32 bytes. Se escribe con `"wb"`.
- Modo **binario** porque la llave son bytes puros; en texto, bytes como saltos de línea o nulos la corrompen.
- Para la práctica la llave quedó **fija**: si el servidor la regenerara en cada arranque (por ejemplo tras Ctrl+C), todas las llaves de los clientes quedarían obsoletas.
- El **IV** (16 bytes) lo genera el cliente en cada ejecución con `RAND_bytes`. Por el IV y el encadenamiento CBC, el mismo texto («Hola Mundo») da bloques cifrados distintos cada vez, lo que evita patrones repetidos en la red.

4. Inspeccionar bytes, bits y ASCII

- Función auxiliar `bytes_to_bits()`: por cada byte imprime decimal, binario y carácter.
- Un byte son 8 bits (0 a 255); con signo, el bit más a la izquierda es el signo y se usa complemento a dos.
- ASCII: 0 a 31 son de control (no imprimibles), 32 es el espacio y desde 33 (`!`) empiezan los imprimibles.

Error común: imprimir el buffer cifrado con `printf("%s")`; se corta en el primer 0x00 o muestra caracteres de control.

5. Integración cliente-servidor (programa_09)

- Antes se probó el par en **texto plano** (`servidor_texto_plano.c` y `cliente_texto_plano.c`, puerto 4898) para ver que un sniffer capturaría el nombre.
- Problema: el servidor necesita la llave (la lee de `llave.key`), el IV del cliente y el tamaño del texto cifrado, que ya no es el del nombre por el relleno.

> **Examen**
>
> Protocolo de **tres `send` y tres `recv`**: (1) el IV de 16 bytes, (2) la longitud del texto cifrado (para reservar memoria), (3) el texto cifrado. «Como el cliente hace tres send, el servidor tiene que hacer tres recv.»
>
> - El servidor descifra, imprime la bitácora (`ID : IP - día/mes/año hora:min:seg - nombre`) y responde «registro exitoso» en texto plano.
> - Compilación: `gcc cliente_texto_crypto.c -o cliente_texto_crypto -lssl -lcrypto` y lo mismo con el servidor. Sin `-lssl -lcrypto` marca error al enlazar.
>
> Error común: descoordinar el orden o la cantidad de `send` y `recv`; el socket se bloquea para siempre.

Contexto — no examen

- Recomendó el libro de criptografía en Java de David Hook para profundizar.
- Casi se tropieza en la tarima y bromeó al respecto.
- En la prueba en vivo varios alumnos se conectaron pero sus nombres no salían; comparó los primeros bytes en pantalla porque todos llegaban por la misma IP de la red PC Puma.

Pendiente

- No hay tarea que entregar. «Tarea moral»: probar en casa el par `servidor_texto_crypto.c` y `cliente_texto_crypto.c` con `llave.key` en el mismo directorio.
- Sigue pendiente de calificar la Tarea 5.
- No hubo pase de lista: la clase se extendió hasta las 8:30. Próxima clase: martes.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

Tema	Idea para recordar
Bloque AES	16 bytes = 32 caracteres hexadecimales
PKCS#7	Faltan k bytes: se agregan k bytes con valor k
Mensaje de 16 bytes	Se agrega un bloque completo de 16 (0x10): total 32
EVP	new → Init_ex → Update → Final_ex; mismo flujo al descifrar
Llave	32 bytes en `llave.key`, abierta con `rb` y `wb`
IV	16 bytes aleatorios por ejecución; mismo texto, distinto cifrado
Protocolo	3 send y 3 recv: IV, longitud y texto cifrado
Compilar	`-lssl -lcrypto`
ASCII	0 a 31 control; 32 espacio; imprimibles desde 33

Hilo conductor

Para cifrar con AES-CBC por un socket hay que rellenar a 16, cifrar con una llave compartida y un IV aleatorio, y enviar IV, longitud y texto en ese orden para que el servidor pueda descifrar.